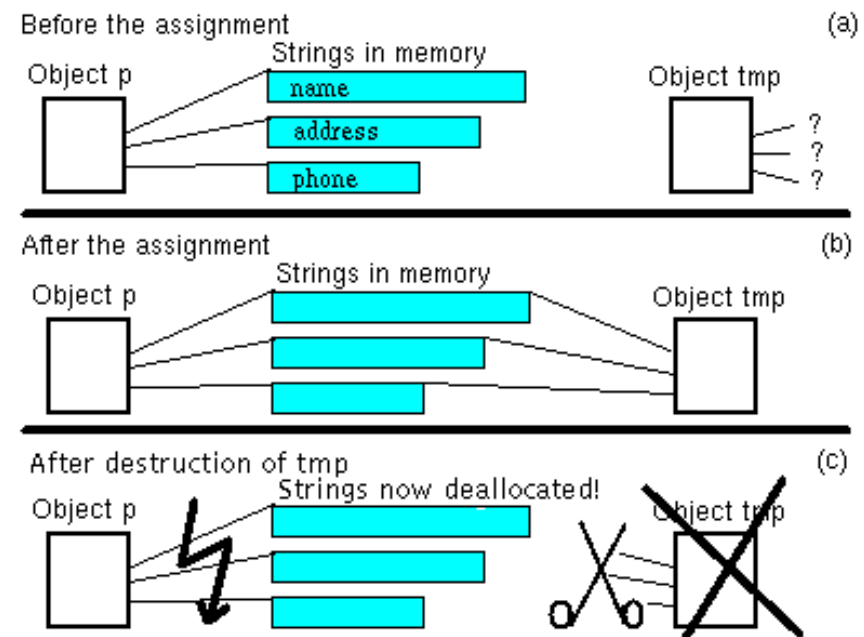


Tilldelnings-operatorn (=)

- I C++ kan vi tilldela värden åt strukturer och klasser precis som vi tilldelar åt heltal etc.

–I praktiken utförs implicit kopiering bit för bit från en struktur till den andra

–Om strukturer innehåller pekare blir et problem, ty →



Copy-constructorn

```
class Person {  
    public:  
        Person(Person const &other) {Copy(other);}  
        Copy(Person const &other);  
}  
void Person::Copy(Person const &other) {  
    delete m_namn;  
    delete m_adress;  
  
    m_namn = strdupnew(other.m_namn);  
    m_adress = strdupnew(other.m_adress);  
}
```

Överladdning av operatorer

- I C++ kan vi tilldela operatorer ny funktionalitet

– t.ex. för assignment =

```
class Person {  
public:  
    void operator=(Person const &other);  
}  
void Person::operator=(Person const &other) {  
    delete m_namn;  
    delete m_adress;  
  
    m_namn = strdupnew(other.m_namn);  
    m_adress = strdupnew(other.m_adress);  
}
```

Överladdning av operatorer

- Vi kan överladda de fleste operatorer, t.ex.

```
class Person {
    char *m_namn;
    char *m_adress;
public:
    int operator>(const Person &other);
    Person(char *, char*);
    Person() {m_namn=m_adress=0;}
};

Person::Person(char *n, char *a) {
    m_namn = strdupnew(n);
    m_adress = strdupnew(a);
}

int Person::operator>(const Person &other) {
    return(strcmp(m_namn, other.m_namn)>0?1:0);
}

main() {
    Person a("axel", "borg"), b("Test", "Addr");
    if (a>b) printf ("a>b\n");
}
```

Överladdning av []-operatorn -exempel

```
class IntArray {
    int *d_data;
    unsigned d_size;
public:
    IntArray(unsigned size = 1);
    IntArray(IntArray const &other);
    ~IntArray();
    IntArray const &operator=(IntArray const &other); // overloaded
    index operators:
    int &operator[](unsigned index); // first
    int const &operator[](unsigned index) const; // second
private:
    void boundary(unsigned index) const;
    void copy(IntArray const &other);
    int &operatorIndex(unsigned index) const;
};
```

Överladdning av []-operatorn -exempel

```
IntArray::IntArray(unsigned size)
: d_size(size) {
    if (d_size < 1) {
        cerr << "IntArray: size of array must be >= 1\n";
        exit(1);
    }
    d_data = new int [d_size];
}

IntArray::IntArray(IntArray const &other) {
    copy(other);
}

IntArray::~~IntArray() {
    delete d_data;
}

IntArray const &IntArray::operator=(IntArray const &other) {
    if (this != &other) {
        delete d_data;
        copy(other);
    }
    return *this;
}
```

Överladdning av []-operatorn -exempel

```
void IntArray::copy(IntArray const &other) {
    d_size = other.d_size;
    d_data = new int [d_size];
    memcpy(d_data, other.d_data, d_size * sizeof(int));
}

int &IntArray::operatorIndex(unsigned index) const {
    boundary(index);
    return d_data[index];
}

int &IntArray::operator[](unsigned index) {
    return operatorIndex(index);
}

int const &IntArray::operator[](unsigned index) const {
    return operatorIndex(index);
}

void IntArray::boundary(unsigned index) const {
    if (index >= d_size) {
        cerr << "IntArray: boundary overflow, index = " <<
            index << ", should range from 0 to " << d_size - 1 << endl;
        exit(1);
    }
}
```

Exempel: Komplexa tal

```
class Complex {  
  
public:  
    // Default constructor + generell constructor  
    Complex(double re=0.0, double img=0.0) {m_re=re;m_img=img;}  
    Complex(const Complex &other) {Copy(other);}  
  
    Complex &operator=(const Complex &other) {Copy(other); return *this;}  
    Complex operator*(const Complex &other) const;  
    Complex operator+(const Complex &other) const;  
    Complex operator/(const Complex &other) const;  
  
    double Re() {return m_re;}  
    double Img() {return m_img;}  
    const char *c_str(); // Formattera som text-sträng  
private:  
    void Copy(const Complex &other) {m_re=other.m_re; m_img=other.m_img;}  
    double m_re;  
    double m_img;  
    char m_c_str[20];  
};
```


Exempel: Komplexa tal 2

```
// (ac-bd) + i(ad+bc)
Complex Complex::operator*(const Complex &other) const {
    Complex res;
    res.m_re = m_re*other.m_re - m_img*other.m_img;
    res.m_img = m_re*other.m_img + m_img*other.m_re;
    return res;
}

// (ac+bd) + i(bc-ad)
// -----
//      c*c+d*d
Complex Complex::operator/(const Complex &other) const {
    Complex res;
    double den = other.m_re*other.m_re + other.m_img*other.m_img;
    res.m_re = (m_re*other.m_re + m_img*other.m_img) / den;
    res.m_img = (m_img*other.m_re - m_re*other.m_img) / den;
    return res;
}

const char *Complex::c_str() {
    sprintf(m_c_str, "%f%s%fj", m_re, m_img<0?"":"+", m_img);
    return m_c_str;
}
```

Exempel: Komplexa tal 3

```
int main(int argc, char* argv[])
{
    Complex b(3.4, 1.2);    // Tal 1
    Complex c(2.2, -5.2);   // Tal 2
    Complex sum = b+c;      // Summan
    Complex prod;           // Produkt
    prod = b*c;             // Beräkna produkt

    printf ("sum of %s and %s is %s\n", b.c_str(), c.c_str(),
            sum.c_str());
    printf ("prod of %s and %s is %s\n", b.c_str(), c.c_str(),
            prod.c_str());
    printf ("div of %s and %s is %s\n", b.c_str(), c.c_str(),
            (b/c).c_str());

    return 0;
}
```

"this"-pekaren

- Då man använder sig av metoder för objekt, har man tillgång till pekaren "this"
 - Pekar alltid på det nuvarande objektet, oberoende om det är på stacken, heapen eller datasegmentet

//T.ex. undvik kopiering av sig själv

```
MyClass &Copy(MyClass const &other) {  
    if (this != &other) {  
        ... // Gör endast om ej självtilldelning  
    }  
    return *this;  
}
```

Operatorer som kan överladdas

+ - * / % ^ & |

~ ! , = < > <= >=

++ -- << >> == != && ||

+= -= *= /= %= ^= &= |=

<<= >>= [] () -> ->* new new[]

delete delete[]

Exceptioner

- Vid felsituationer i C, sker ofta något av följand
 - Funktionen noterar det ovanliga, och ger ett meddelande
 - Funktionen stoppar exekveringen och returnerar ett felmeddelande till kallaren
 - Den kallande funktionen kan i sin tur föra felmeddelanden uppåt i hierarkin
 - Funktionen gör beslutet att saker spårar ur och avslutar processen med t.ex. `exit()`
 - Funktionen använder en kombination av `setjmp()` och `longjmp()`, för att direkt återvända till en högre nivå

Exceptioner i C++

- C++ erbjuder naturligtvis alla föregående sätt att hantera felsituationer
- C++ erbjuder dessutom exceptioner, eller sätt att kontrollerat återvända i call-stacken till en högre nivå
- När funktionen själv inte kan hantera felet på ett vettigt sätt, är exceptioner ett gott alternativ

Exceptioner: Syntax

```
try {  
    // kod, inom vilken exceptioner kan  
    // genereras  
} catch (<type> <var>) {  
    // Vad som händer, om en exception av  
    // given typ genereras  
}  
  
// Exceptioner genereras genom  
throw "This generates a char * exception;
```

Exceptioner: hierarki

- try-block kan finnas på olika nivåer, t.ex.
 - main(): huvudnivå, hanterar exceptioner om inte andra hanterar hittas
 - i funktioner: beroende på funktion, kan denna hantera exceptionerna

```
int main() {  
    try {  
        myfunc1();  
        myfunc2();  
    } catch (...) {  
        printf("Exception\n");  
    }  
}  
void myfunc1() {  
    try {  
        // Egen hanterare  
    } catch(int i) { //kod }  
}
```


Exceptioner: default catcher

```
try {  
    // Kod  
} catch (...) { //Default catcher  
    printf("Nåt fel uppstod (vad sägs om out of  
memory??\n");  
}  
  
try {  
    try {  
        throw 12.33;  
    } catch (char *s) {  
        printf ("Char * exception\n");  
    } catch (...) {  
        printf("Generic error handler\n");  
        throw;  
    }  
} catch (double d) {  
    printf("Outer level double exception catcher\n");  
}  
}
```

Exceptioner som inte fångas

- En exception som inte "fångas" går till
 - `std::terminate()`
- `std::terminate()` kallas i sin tur på
 - `std::abort()`

Exceptioner i konstruktörer

- Om man genererar en exception i en konstruktor så att exceptionen sker inne i konstruktorn, kommer destruktorn inte att kallas
 - Om man allokerat minne före denna "förlorade" exception, kommer en minnesläcka att uppstå
- Regel: En exception som genereras i en konstruktor skall inte okontrollerat gå vidare upp i hierarkin

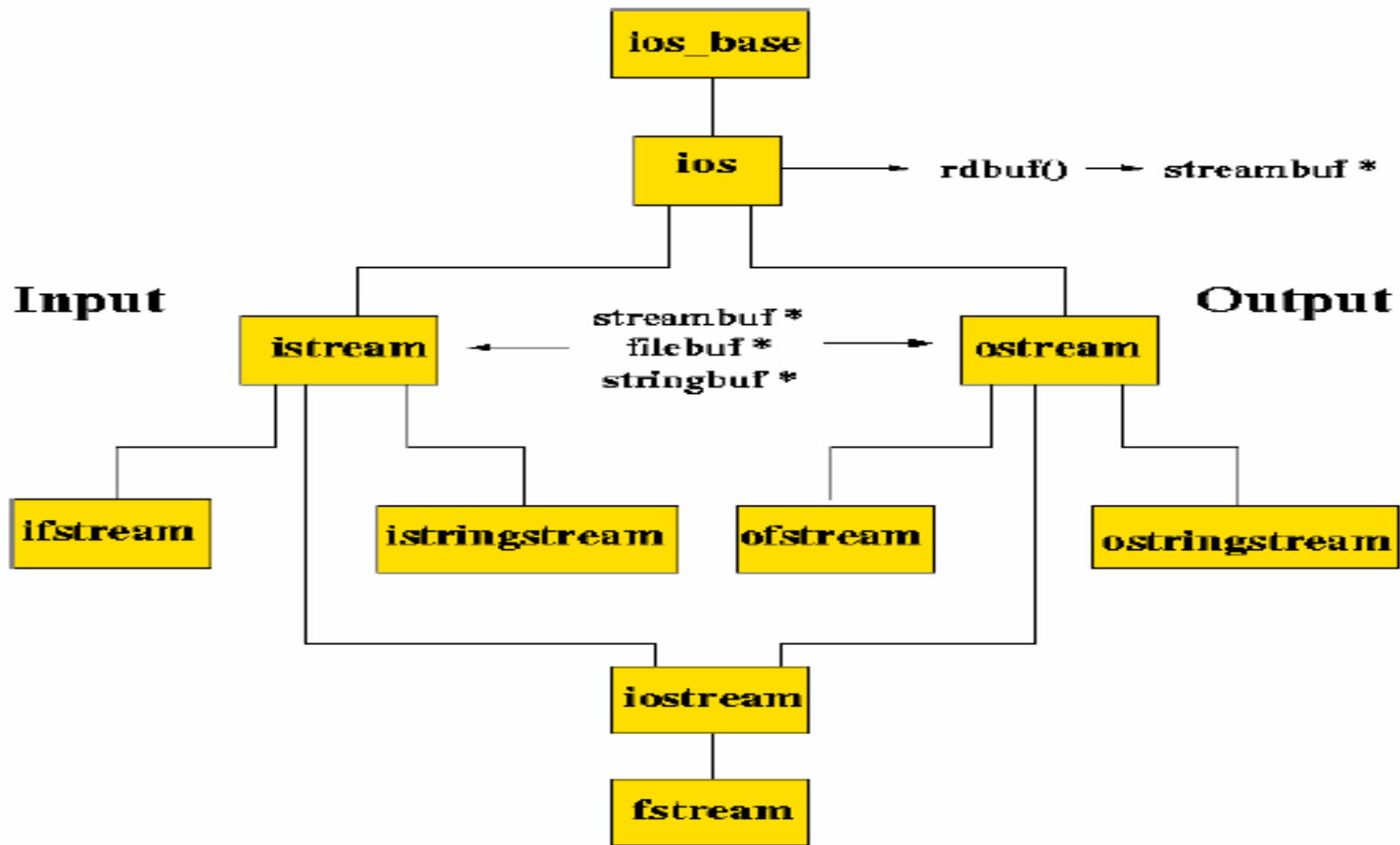
iostream-biblioteket

- I C++ definieras i/o-strömmarna **cout**, **cin**, **cerr** genom att inkludera filen `<iostream>`

```
#include <iostream>
main() {
    int nval;
    cout << "Ge ett nummer" <<endl;
    cin >> nval;
    cout << "Numret du gav var: " << nval << endl;
}
```

- **cout**, **cin**, **cerr** är de fakto objekt-instanser av motsvarande klasser, "<<" och ">>" är överladdade operatorer

Centrala i/o-klasser



iostream....

- Genom att använda överladdade operatorer kan man "mata in" olika variabler till stream-biblioteket

```
#include <iostream>

main() {
    int nval;
    float f;
    double d;
    cout << "Ge ett nummer" << endl;
    cin >> nval;
    f = nval; d = nval;
    cout << "Numret du gav var: " << nval << endl;
    cout << "Flyttal: " << f << endl;
    cout << "Dubbel prec.: " << d << endl;
}
```

iostream

- Ger säkerhet till i/o, jfr C: printf

```
double bt = (double)133/(double)5;
printf("Bilar per timme %i\n", bt);
```

– ger svaret –1717986918, p.g.a. fel i i
matchingen format <-> parametrar

```
cout << "Bilar per timme " <<
(double)133/(double)5<<endl;
```

ger däremot rätt svar 26.6

iostream-formattering -utdrag

- `ios::adjustfield`:
 - mask value used in combination with a flag setting defining the way values are adjusted in wide fields (`ios::left`, `ios::right`, `ios::internal`). Example, setting the value 10 left-aligned in a field of 10 character positions:

```
cout.setf(ios::left, ios::adjustfield);  
cout << "'" << setw(10) << 10 << "'" << endl;
```

- `ios::basefield`:
 - mask value used in combination with a flag setting the radix of integral values to output (`ios::dec`, `ios::hex` or `ios::oct`). Example, printing the value 57005 as a hexadecimal number:

```
cout.setf(ios::hex, ios::basefield);  
cout << 57005 << endl;
```

```
// or, using the manipulator:
```

```
cout << hex << 57005 << endl
```

OBS! Mera i t.ex. C++-annotations

Utskrift till fil

```
#include <iostream>
int main()
{
    ofstream of;
    cout << "of's open state: " << boolalpha <<
        of.is_open() << endl;
    of.open("/tmp/myfile"); // on Unix systems
    cout << "of's open state: " << of.is_open() <<
        endl;
    of << "Testing output to file" << (float)5.44;

}
```

Generisk programmering

- C++ erbjuder möjligheter att definiera och implementera generella (eller abstrakta) funktioner och klasser
- Detta system kallas *Template Mechanism*
- Genom att specificera ett template, kan kompilatorn generera kod, som är anpassad till den egentliga datatyp som används
- Template-mekanismen kan ses som jobbig i början, men efterhand brukar man se nytta av den

Generisk programmering: intro

```
// Generell funktion: Addera två argument
Type add(Type const &lvalue, Type const &rvalue) {
    return lvalue + rvalue;
}
//För type double
double add(double const &lvalue, double const
    &rvalue) {
    return lvalue + rvalue;
}
```

- Detta kan förstås upprepas för varje tänkbar datatyp, användande av överladdade funktioner... men leder till ...många... versioner av funktionen

Generisk funktion

```
template <typename Type>
Type add(Type const &lvalue, Type const
        &rvalue) {
    return lvalue + rvalue;
}
```

```
// Nu kan vi använda
double a=1.2, b=3.34;
printf("sum of %f and %f is %f\n", a, b,
        add(a,b));
```

```
// Kompilatorn genererar nu en överladdad
funktion av rätt typ
```

Format på template-funktion

Keyword

*Kommaseparerad lista med templat-
parameterlista*

template <typename Type>

```
Type add(Type const &lvalue, Type const  
    &rvalue) {  
    return lvalue + rvalue;  
}
```

Vi kan notera, att hittills har vi endast använt formella variabelnamn som argument till funktioner, typerna har varit givna.

Nu låter vi även typerna på argumenten fungera som formella namn.

Exempel med den komplexa datatypen

```
template <typename Type>
Type add(Type const &lvalue, Type const
        &rvalue) {
    return lvalue + rvalue;
}

int main() {
    cout << "Sum of " << x << " and " << y << " =
    " << add(b,c) <<endl;
}
```

Hur avgörs argumenttyp??

- Endast funktionargument används, inte lokala variabler eller retur-värden
- 3(4) konversioner kan användas
 - lvalue till rvalue
 - tilläggande av const till icke-const
 - konversion till bas-klass
 - standard konversioner (int to double, int to unsigned etc)

Template-klasser

- Används i STL (Standard Template Library)
- Typisk för klasser som enkapsulerar andra datatyper såsom
 - Vektorer
 - Matriser
 - Länkade listor

Kompleksa klassen med templat

```
template <typename T1>
class Complex {
public:
    Complex(T1 re=0.0, T1 img=0.0) {m_re=re;m_img=img;}
    Complex(const Complex &other) {Copy(other);}

    Complex &operator=(const Complex &other) {Copy(other); return *this;}
    Complex operator*(const Complex &other) const;
    Complex operator/(const Complex &other) const;
    Complex operator+ (const Complex &other) const {Complex res(*this);
    res.m_re+=other.m_re;res.m_img+=other.m_img; return res;}
    //Complex const operator+(const Complex &other) {Complex res(*this);
    res.m_re+=other.m_re;res.m_img+=other.m_img; return res;}
    int a;
    //ostream &operator<<(ostream &stream, Complex const &c);

    T1 const Re() {return m_re;}
    T1 const Img() {return m_img;}
    const char *c_str();
private:
    void Copy(const Complex &other) {m_re=other.m_re; m_img=other.m_img;}
    double m_re;
    double m_img;
    char m_c_str[20];
};
```

Komplexa tal 2

```
// (ac-bd) + i(ad+bc)
template <typename T1>
Complex<T1> Complex<T1>::operator*(const Complex<T1> &other) const {
    Complex res;
    res.m_re = m_re*other.m_re - m_img*other.m_img;
    res.m_img = m_re*other.m_img + m_img*other.m_re;
    return res;
}
// (ac+bd) + i(bc-ad)
// -----
//      c*c+d*d
template <typename T1>
Complex<T1> Complex<T1>::operator/(const Complex<T1> &other) const{
    Complex<T1> res;
    double den = other.m_re*other.m_re + other.m_img*other.m_img;
    res.m_re = (m_re*other.m_re + m_img*other.m_img) / den;
    res.m_img = (m_img*other.m_re - m_re*other.m_img) / den;
    return res;
}
template <typename T1>
const char *Complex<T1>::c_str() {
    sprintf(m_c_str, "%f%s%fj", m_re, m_img<0?"":"+", m_img);
    return m_c_str;
}
```

Komplexa 3

```
typedef Complex<double> complex; // Komplex klass med double som element
//typedef Complex<int> complex; // Komplex klass med heltal som element

int main(int argc, char* argv[])
{
    complex b(3.4, 1.2);
    complex c(2.2, -5.2);
    complex sum = b+c;
    complex prod;
    prod = b*c;
    double x=1.22, y=4.55;

    using namespace std;

    printf ("sum of %s and %s is %s\n", b.c_str(), c.c_str(), sum.c_str());
    printf ("prod of %s and %s is %s\n", b.c_str(), c.c_str(), prod.c_str());

    printf ("div of %s and %s is %s\n", b.c_str(), c.c_str(), (b/c).c_str());

    cout << "Sum of " << x << " and " << y << " = " << add(b,c) <<endl;
}
```

Ex: Stack

```
template <class T>
class Stack {
public:
    Stack(int = 10) ;
    ~Stack() { delete [] stackPtr ; }
    int push(const T&);
    int pop(T&) ;
    int isEmpty()const { return top == -1 ; }
    int isFull() const { return top == size - 1 ; }
private:
    int size ; // number of elements on Stack.
    int top ;
    T* stackPtr ; } ;
```

Ex: Stack (2)

```
//constructor with the default size 10
template <class T>
Stack<T>::Stack(int s) {
    size = s > 0 && s < 1000 ? s : 10 ;
    top = -1 ; // initialize stack
    stackPtr = new T[size] ;
}
// push an element onto the Stack
template <class T>
int Stack<T>::push(const T& item) {
    if (!isFull()) {
        stackPtr[++top] = item ;
        return 1 ; // push successful
    }
    return 0 ; // push unsuccessful
}
```

Ex: Stack (3)

```
// pop an element off the Stack
template <class T>
int Stack<T>::pop(T& popValue) {
    if (!isEmpty()) {
        popValue = stackPtr[top--] ;
        return 1 ; // pop successful
    }
    return 0 ; // pop unsuccessful
}
```

Ex: Stack (4)

```
#include <iostream>
#include "stack.h"
using namespace std ;
void main() {
    typedef Stack<float> FloatStack ;
    typedef Stack<int> IntStack ;
    FloatStack fs(5) ;
    float f = 1.1 ;
    cout << "Pushing elements onto fs" << endl ;
    while (fs.push(f)) { cout << f << ' ' ; f += 1.1 ; }
    cout << endl << "Stack Full." << endl << endl << "Popping elements
    from fs" << endl ;
    while (fs.pop(f))
        cout << f << ' ' ; cout << endl << "Stack Empty" << endl ;
    cout << endl ;
    IntStack is ;
    int i = 1.1 ;
    cout << "Pushing elements onto is" << endl ;
    while (is.push(i)) { cout << i << ' ' ; i += 1 ; }
    cout << endl << "Stack Full" << endl << endl << "Popping elements
    from is" << endl ;
    while (is.pop(i)) cout << i << ' ' ; cout << endl << "Stack Empty"
    << endl ;
}
```

This document was created with Win2PDF available at <http://www.win2pdf.com>.
The unregistered version of Win2PDF is for evaluation or non-commercial use only.